

Tokens: JWT, OAuth 2.0 e OIDC

Autenticação Moderna em Aplicações AWS

AWS Certified Developer Associate

Instrutor: Marlon Anesi

JSON Web Token (JWT): Estrutura e Uso

JWT = token autocontido, assinado digitalmente. Permite verificar autenticidade sem consultar banco de dados. Formato: Base64(Header).Base64(Payload).Signature

Estrutura JWT (3 partes separadas por ponto):

Header

```
{ "alg": "RS256", "typ": "JWT" }
```

Algoritmo de assinatura e tipo do token. RS256 = RSA + SHA-256 (assimétrico). HS256 = HMAC (simétrico).

Payload (Claims)

```
{ "sub": "1234", "email": "user@x.com", "exp":  
1700000000, "iss": "cognito.aws.com" }
```

sub=subject(id do usuário). exp=expiração. iss=issuer. aud=audience. Pode ter claims customizados.

Signature

```
HMACSHA256(base64(header) + '.' + base64(payload),  
secret)
```

Garante integridade. Verificada sem banco de dados. Invalida se payload for alterado.

CRÍTICO: JWT é stateless. Servidor verifica assinatura localmente, sem consulta ao IdP. Invalide tokens via exp e blacklist se necessário.

OAuth 2.0: Framework de Autorização

OAuth 2.0 = framework de AUTORIZACAO. Permite que apps obtenham acesso limitado a recursos em nome do usuário. NÃO é autenticação por si só.

Grant Types (Fluxos de Autorização):

Authorization Code MAIS SEGURO	App redireciona para IdP. Usuário faz login. IdP retorna code. App troca code por access token no backend. Usado em web/mobile apps.
Authorization Code + PKCE MOBILE/SPA	Extension do Authorization Code para apps sem backend seguro (mobile, SPA). PKCE previne interceptação do code.
Client Credentials MACHINE-TO-MACHINE	App usa client_id + client_secret para obter token. Sem usuário envolvido. Para serviços de backend.
Implicit (Deprecated) EVITAR	Token retornado diretamente na URL. Obsoleto e inseguro. Não usar em novas aplicações.

OpenID Connect (OIDC): Autenticação sobre OAuth 2.0

OIDC = camada de autenticação SOBRE o OAuth 2.0. Adiciona o ID Token (JWT com claims do usuário). OAuth: AUTORIZACAO. OIDC: AUTENTICACAO.

Comparativo:

	OAuth 2.0	OIDC
Objetivo	Autorizacao (acesso a recursos)	Autenticacao (identidade do usuario)
Token emitido	Access Token (opaco ou JWT)	ID Token (JWT) + Access Token
Quem e o usuario?	NAO responde isso	SIM: claims no ID Token
Uso no AWS	Client Credentials, machine-to-machine	Cognito, AssumeRoleWithWebIdentity

Endpoints OIDC (usados por Cognito e outros IdPs):

```
/authorize - Inicia o fluxo de autenticação
/token     - Troca authorization code por tokens
/userinfo  - Retorna claims do usuário autenticado
/.well-known/openid-configuration - Discovery document (todos os endpoints)
```

Bearer Tokens e Casos de Uso no Exame

Proteger API com JWT (Cognito)

- API Gateway usa Cognito Authorizer
- Valida ID Token ou Access Token
- Sem código no Lambda para verificar auth
- Claims usados para controle de acesso

Machine-to-Machine (M2M)

- Use OAuth Client Credentials grant
- Sem usuário: app obtém token diretamente
- Escopo define o que o serviço pode fazer
- Ex: microservice A chamando B

Como Usar um Bearer Token (Header HTTP):

```
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0IiwiaWwiOiJ1c2VyQGV4YW1wbGUuY29tIiwiaXhwIjoxNzAwMDAwMDAwfQ.signature
```

- Nunca envie tokens em query strings (URL pode ser logada). Sempre use Authorization header.
- Verifique o campo exp (expiração) antes de usar o token. Token expirado deve ser rejeitado.
- Valide o issuer (iss) e audience (aud) para prevenir token substitution attacks.

Pontos-chave para o Exame

1

JWT e autocontido e stateless

Servidor verifica a assinatura localmente. Sem banco de dados. Se token for valido e não expirado -> acesso permitido.

2

OAuth = autorização, OIDC = autenticação

OAuth 2.0 controla o QUE pode ser acessado. OIDC adiciona ID Token com QUEM e o usuário (claims de identidade).

3

Authorization Code com PKCE para mobile/SPA

Mais seguro para clientes públicos. PKCE previne que o code seja interceptado e usado por terceiros.

LEMBRE-SE: OAuth = AUTORIZACAO. OIDC = AUTENTICACAO. JWT expirou? Renovar com Refresh Token. M2M? Client Credentials!